



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

Aprendizaje automático - Sesión 6

Bagging and Boosting

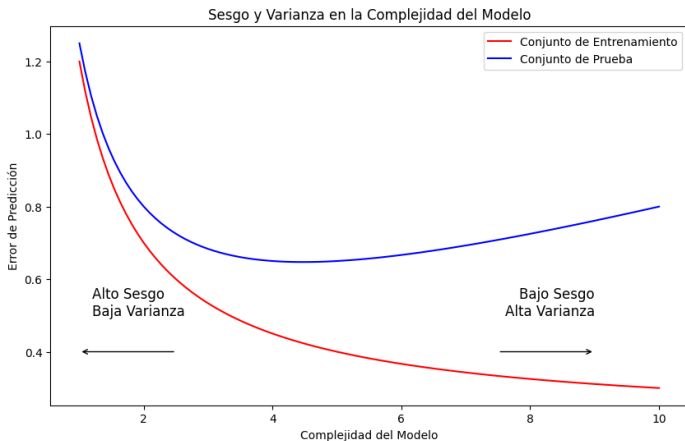
Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.
jorge.portilla@usm.cl

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

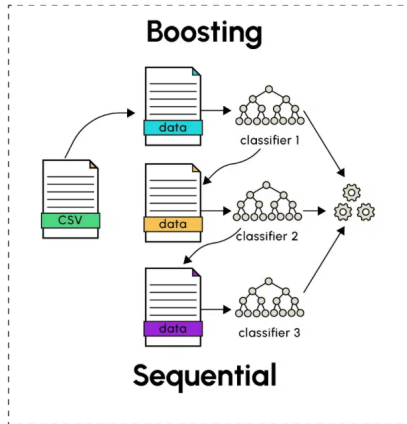
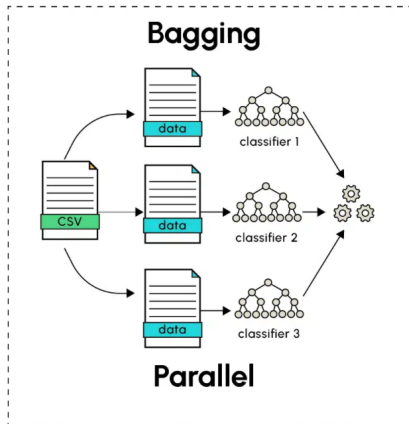
- ▶ **Bias:** Error debido a suposiciones demasiado fuertes del modelo. **Ocorre cuando el modelo es muy simple** para capturar la complejidad de los datos.
- ▶ **Variance:** Error causado por la sensibilidad del modelo a pequeñas variaciones en los datos de entrenamiento. Modelos muy complejos tienden a sobreajustarse.
- ▶ **Noise:** Error inherente a los datos. Es irreducible y no depende del modelo.
- ▶ **Bagging y Boosting:**
 - **Bagging** reduce la varianza.
 - **Boosting** reduce el sesgo y también puede ayudar con la varianza.

Sesgo y varianza



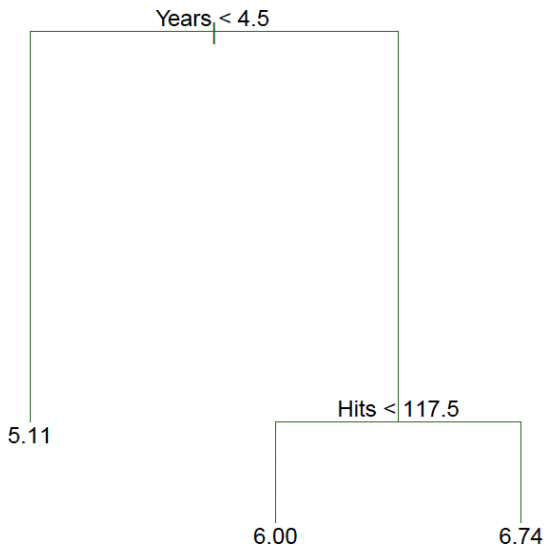
- Relación **sesgo y varianza** está relacionado con la complejidad del modelo.
- Modelo con baja complejidad tiene un **alto sesgo** al cubrir los puntos de entrenamiento (**underfitting**).
- Modelo con **alta complejidad**, la predicción tiende a ser más cercana (menor sesgo) a los datos de entrenamiento. (**overfitting**).

Bagging & Boosting



- ▶ *These involve **stratifying** or **segmenting** the predictor space into a number of simple regions.*
- ▶ Para predecir una observación, se usa el promedio de las respuestas en la región correspondiente.
 - Estas regiones se definen con **reglas de división** que se representan en forma de árbol, por eso se llaman métodos de **árboles de decisión**.
- ▶ Los métodos basados en árboles son simples y fáciles de interpretar, pero suelen tener menor precisión que otros métodos avanzados.
 - Por eso, se usan enfoques como **bagging, random forests y boosting**, que combinan muchos árboles para mejorar la predicción.
 - Esta combinación suele aumentar la precisión, aunque se pierde algo de interpretabilidad.

Regression trees



- Árbol de regresión para predecir $\log(\text{salario})$ usando Years y Hits.
- Cada nodo divide los datos; las hojas muestran el promedio de $\log(\text{salario})$.

Example: Regression tree with Hitters

- ▶ Se construye un árbol de regresión para predecir el **log(salario)** de jugadores.
- ▶ El árbol realiza divisiones sucesivas basadas en dos variables:
 - Years: años jugados en las Grandes Ligas.
 - Hits: hits obtenidos el año anterior.
- ▶ El árbol genera tres regiones:

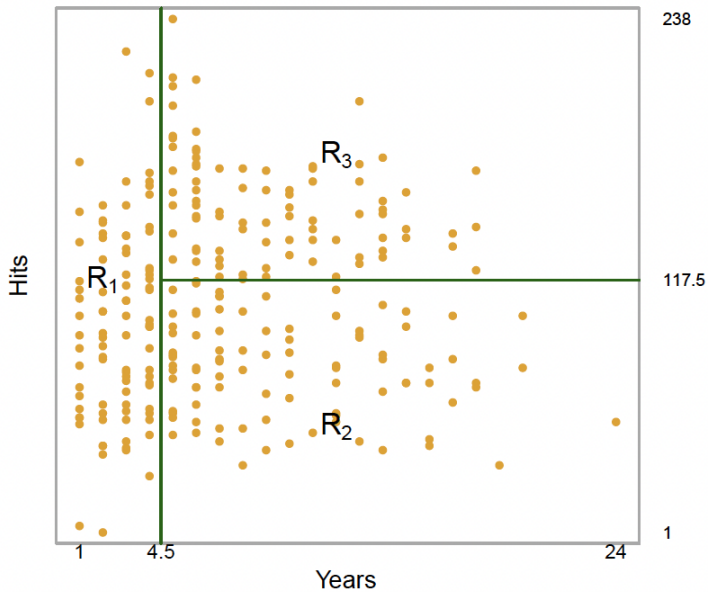
$$R_1 = \{X \mid \text{Years} < 4,5\}$$

$$R_2 = \{X \mid \text{Years} \geq 4,5, \text{Hits} < 117,5\}$$

$$R_3 = \{X \mid \text{Years} \geq 4,5, \text{Hits} \geq 117,5\}$$

- ▶ Predicciones de salario por grupo (en miles de dólares):
 - $R_1: e^{5,107} \approx \$165,174$
 - $R_2: e^{5,999} \approx \$402,834$
 - $R_3: e^{6,740} \approx \$845,346$
- ▶ Cada hoja del árbol representa el **promedio del log-salario** para los jugadores en esa región.

Regression trees



Estructura e interpretación del árbol de regresión

- ▶ En el árbol, las divisiones internas se llaman **nodos internos** (ej. $\text{Years} < 4.5$, $\text{Hits} < 117.5$).
- ▶ Las **hojas** o **nodos terminales** representan regiones finales del espacio de predictores (R_1 , R_2 , R_3).
- ▶ Las conexiones entre nodos se llaman **ramas**.
- ▶ Interpretación del árbol:
 - Years es el factor más importante: jugadores con menos experiencia ganan menos.
 - Para jugadores con $\text{Years} < 4.5$, los Hits tienen poco impacto en el salario.
 - Entre jugadores con más experiencia ($\text{Years} \geq 4.5$), los Hits sí influyen: más hits $\rightarrow$ mayor salario.
- ▶ A pesar de que el árbol tiende a simplificar: **representación gráfica que facilita la interpretación.**

A continuación, se describe el proceso de construcción de un árbol de regresión. A grandes rasgos, consta de dos pasos:

- Se divide el espacio de predictores (es decir, el conjunto de posibles valores para $X_1, X_2, \dots, X_p$) en J regiones distintas y no superpuestas: $R_1, R_2, \dots, R_J$.
- Para cada observación que cae en una región R_j , se realiza la misma predicción, que consiste simplemente en el promedio de los valores de las observaciones de entrenamiento $\in R_j$.

Ejemplo

Supongamos que en el **Paso 1** obtenemos dos regiones, R_1 y R_2 , y que la media de la respuesta de las observaciones en R_1 es 10, mientras que en la segunda es 20. Entonces, para una observación dada $X = x$:

- ▶ Si $x \in R_1$, se predice un valor de 10.
- ▶ Si $x \in R_2$, se predice un valor de 20.

Ejemplo

Supongamos que en el **Paso 1** obtenemos dos regiones, R_1 y R_2 , y que la media de la respuesta de las observaciones en R_1 es 10, mientras que en la segunda es 20. Entonces, para una observación dada $X = x$:

- ▶ Si $x \in R_1$, se predice un valor de 10.
- ▶ Si $x \in R_2$, se predice un valor de 20.

¿Cómo se construyen las regiones $R_1, \dots, R_J$?

- ▶ En teoría, las regiones pueden tener cualquier forma.
- ▶ Sin embargo, por simplicidad e interpretabilidad, se dividen como rectángulos de alta dimensión (cajas).
- ▶ El objetivo es encontrar cajas $R_1, \dots, R_J$ que minimicen la suma de errores cuadrados residuales (RSS), dada por:

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

donde $\hat{y}_{R_j}$ es la media de las respuestas en la caja j .

- ▶ Considerar todas las posibles particiones del espacio predictor es computacionalmente inviable.
- ▶ Por esto, usamos un método de división binaria recursiva.

División binaria recursiva:

- ▶ Es un enfoque **top-down**: comienza con todos los datos en una sola región y va dividiendo sucesivamente.
- ▶ En cada paso elige la mejor división local sin mirar divisiones futuras.

Recursive Binary Splitting

Para realizar la **división binaria recursiva**, se siguen estos pasos:

- Se selecciona el predictor X_j y un punto de corte s tal que dividir el espacio de predictores en:

$$R_1(j, s) = \{X \mid X_j < s\}, \quad R_2(j, s) = \{X \mid X_j \geq s\}$$

produce la mayor reducción posible en la **suma de errores cuadráticos residuales (RSS)**.

- Se consideran todos los predictores $X_1, \dots, X_p$ y todos los posibles valores de s para cada uno.
- Se elige la combinación (j, s) que minimiza:

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

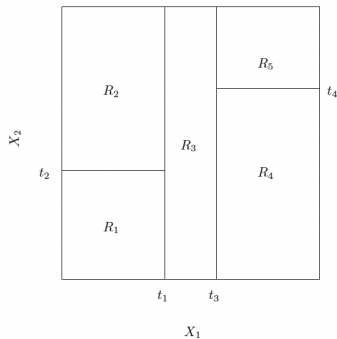
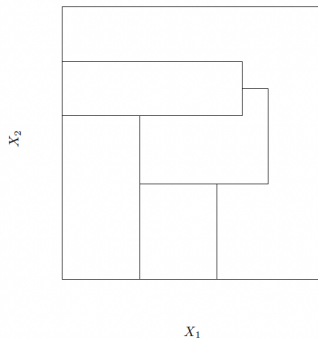
donde $\hat{y}_{R_1}$ y $\hat{y}_{R_2}$ son las medias de las respuestas en las regiones R_1 y R_2 respectivamente.

- Esta búsqueda se puede realizar de forma eficiente, especialmente si el número de predictores p no es muy grande.

Tree construction: Iterative process

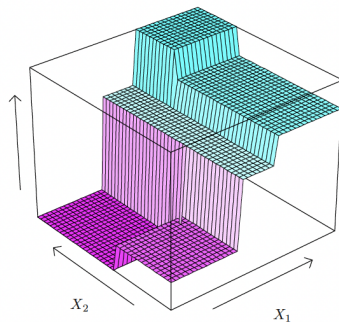
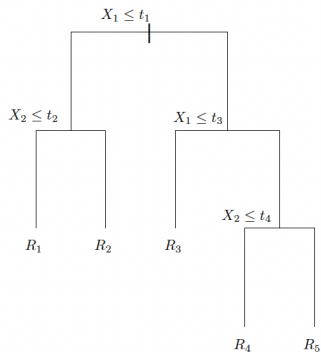
- ▶ El proceso de división continúa de forma iterativa:
 - Se busca nuevamente el mejor predictor y punto de corte para dividir los datos y minimizar el RSS.
 - Esta vez, se divide **una de las regiones ya existentes**, no todo el espacio de predictores.
- ▶ Así, se generan más regiones: primero tres, luego cuatro, y así sucesivamente.
- ▶ Este procedimiento continúa hasta cumplir un **criterio de detención**, por ejemplo:
 - **Detenerse cuando ninguna región contiene más de cinco observaciones.**
- ▶ Finalmente, cuando se han creado las regiones $R_1, \dots, R_J$, la predicción para una observación de prueba se realiza como:
 - La **media** de las respuestas de entrenamiento en la región a la que pertenece dicha observación.

División binaria recursiva



Izquierda: Una partición del espacio bidimensional de predictores que **no puede** resultar de la división binaria recursiva. **Derecha:** Resultado de la división binaria recursiva en un ejemplo bidimensional.

Árbol de regresión



Izquierda: Árbol correspondiente a la partición mostrada en la fig. superior derecha anterior. **Derecha:** Superficie de predicción correspondiente al árbol mostrado.

Problema

El árbol de regresión construido puede sobreajustar los datos, generando:

- ▶ Buen desempeño en el conjunto de entrenamiento.
- ▶ Mal desempeño en el conjunto de prueba.

Tree pruning

- ▶ En lugar de detener el crecimiento del árbol temprano, se recomienda:
 - Construir un árbol grande T_0 .
 - **Podarlo para obtener un subárbol con menor error de prueba.**
- ▶ ¿Cómo determinar qué subárbol elegir?
 - Mediante validación cruzada o conjunto de validación.
 - Evaluar solo un conjunto reducido de subárboles candidatos (no todos).

Cost Complexity Pruning

En lugar de evaluar todos los subárboles posibles, consideramos una secuencia de subárboles indexados por un parámetro $\alpha \geq 0$.

Función objetivo

Para cada α , buscamos el subárbol $T \subseteq T_0$ que minimice:

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

donde:

- ▶ $|T|$ es el número de nodos terminales en el árbol T ,
- ▶ R_m es la región correspondiente al nodo terminal m ,
- ▶ $\hat{y}_{R_m}$ es la media de las observaciones en R_m .

Interpretación

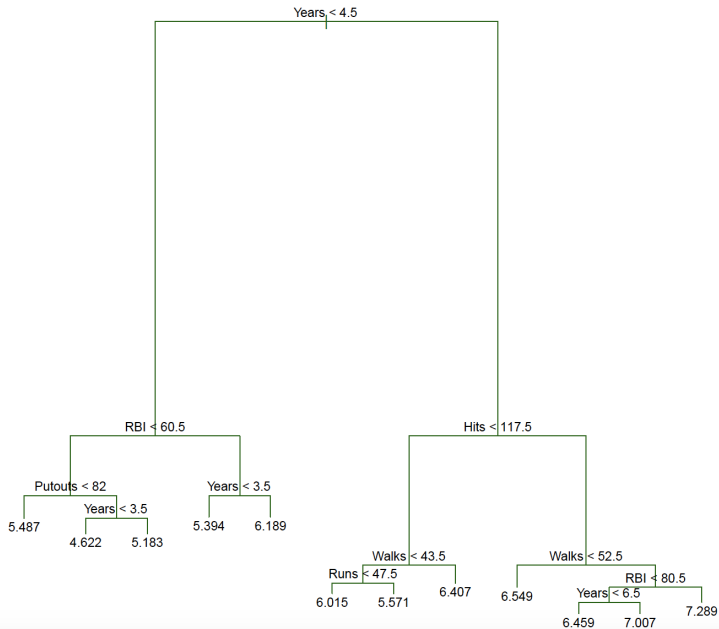
- ▶ α controla el balance entre complejidad y ajuste.
- ▶ $\alpha = 0 \Rightarrow$ árbol completo T_0 .
- ▶ Al aumentar α , se penalizan árboles más complejos, favoreciendo subárboles más pequeños.

Algoritmo — Construcción de un Árbol de Regresión

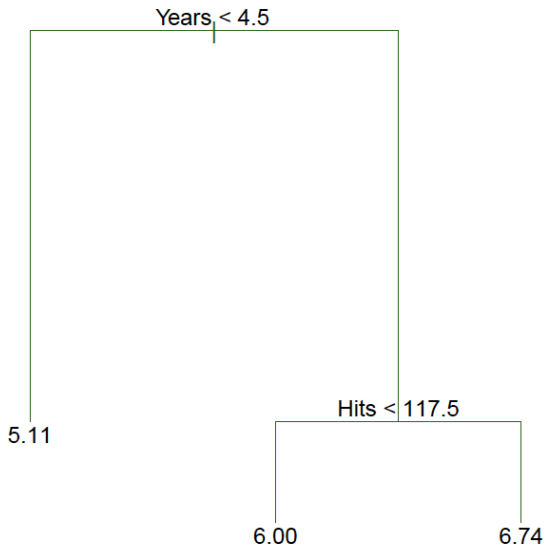
1. **Crecimiento del árbol completo:** Utilizar **división binaria recursiva** para construir un árbol grande sobre el conjunto de entrenamiento¹.
2. **Poda por complejidad de costo:** Aplicar poda para obtener una secuencia de subárboles óptimos según el parámetro α .
3. **Selección de α mediante validación cruzada:**
 - 1) Dividir el conjunto de entrenamiento en K particiones (folds).
 - 2) Para cada $k = 1, \dots, K$:
 - Repetir los pasos 1 y 2 con todos los datos excepto el fold k .
 - Evaluar el error cuadrático medio de predicción en el fold k , como función de α .
 - 3) Promediar los errores para cada valor de α y seleccionar el que minimiza el error medio.
4. **Árbol final:** Retornar el subárbol de la secuencia (**Paso 2**) correspondiente al α óptimo.

¹Detenerse solo cuando cada nodo terminal tenga menos que un número mínimo de observaciones.

Árbol sin podar

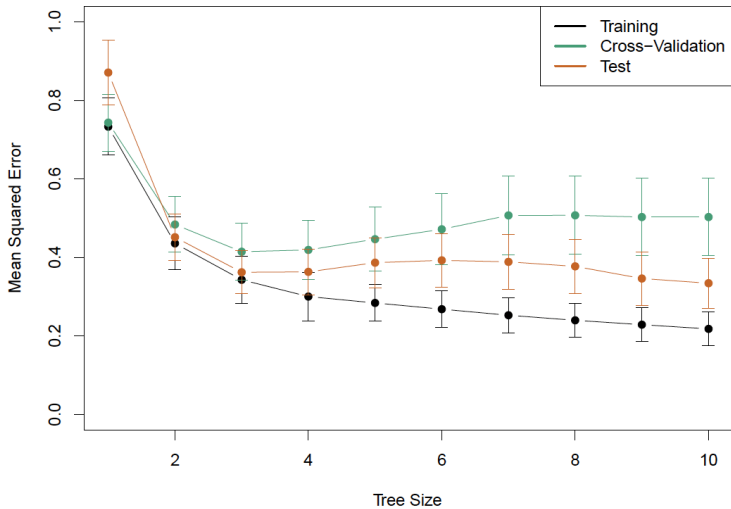


Árbol con poda



Resultados del Árbol de Regresión en el Conjunto Hitters

- ▶ Se utilizaron nueve variables para predecir el salario.
- ▶ El conjunto de datos se dividió aleatoriamente en mitades:
 - **Entrenamiento:** 132 observaciones
 - **Prueba:** 131 observaciones
- ▶ Se ajustó un árbol de regresión grande y se aplicó **poda por complejidad de costo** (α en la ecuación **Slide 18**).
- ▶ Se usó **validación cruzada de 6 folds** para estimar el MSE como función de α .
- ▶ **Figura Slide 22:**
 - Curva verde: error CV vs. número de hojas
 - Curva naranja: error en conjunto de prueba
 - Curva negra: error en conjunto de entrenamiento
 - Mínimo CV en árbol con 3 nodos terminales
- ▶ El árbol podado con tres nodos se muestra en la Figura **Slide 20**.



Classification Trees

- ▶ Son similares a los árboles de regresión, pero predicen una **respuesta cualitativa**.
- ▶ En cada nodo terminal, se predice la **clase más frecuente** entre las observaciones de entrenamiento en esa región.
- ▶ Además del valor de clase, interesa conocer la **proporción de clases** en cada nodo.
- ▶ El crecimiento del árbol se realiza mediante **división binaria recursiva**, igual que en regresión.
- ▶ Sin embargo, el **RSS no es aplicable**, ya que se trata de una respuesta categórica.

Criterios para Crecimiento de Árboles de Clasificación

- Una medida natural es el **tasa de error de clasificación**:

$$E = 1 - \max_k \hat{p}_{mk}$$

donde $\hat{p}_{mk}$ es la proporción de observaciones en la región m que pertenecen a la clase k .

- Sin embargo, esta métrica **no es suficientemente sensible** para construir árboles.
- Se prefieren medidas más finas como el:

Índice de Gini

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

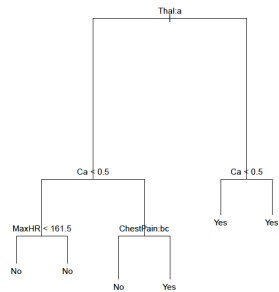
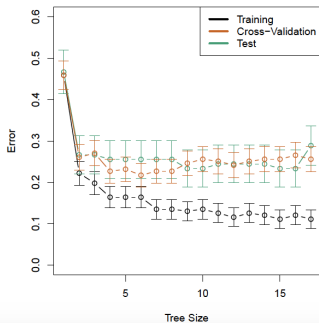
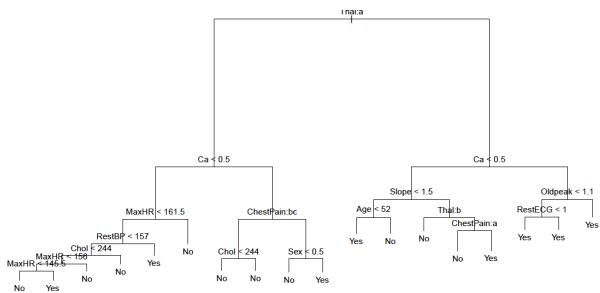
- El índice de Gini mide la **impureza del nodo**: valores pequeños indican que la mayoría de las observaciones pertenecen a una sola clase.

- Una alternativa al índice de Gini es la **entropía**:

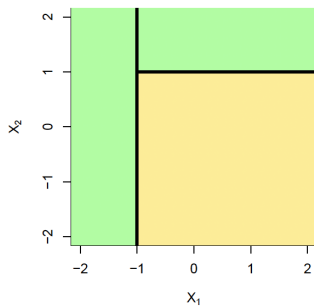
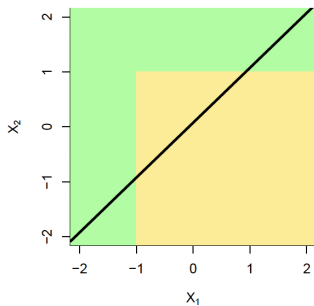
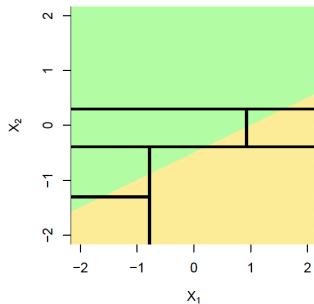
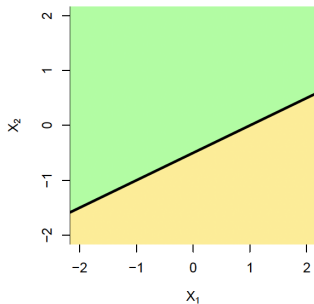
$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

- Se basa en la teoría de la información y mide la **incertidumbre** en una distribución de clases.
- La entropía es cercana a cero cuando el nodo es **puro** (una clase dominante).
- Tanto la entropía como el índice de Gini son más **sensibles a la pureza del nodo** que la tasa de error de clasificación.
- Para **crecer el árbol**, se suele usar entropía o Gini; para **podar el árbol**, se puede usar la tasa de error si se busca minimizar el error de predicción final.

- ▶ El dataset *Heart* contiene información de **303 pacientes con dolor torácico**.
- ▶ Variable respuesta binaria HD:
 - Yes = presencia de enfermedad cardíaca.
 - No = ausencia de enfermedad cardíaca.
- ▶ Hay **13 predictores**, tanto continuos como cualitativos:
 - Ej. continuos: Age, Chol (colesterol).
 - Ej. cualitativos: Sex, Thal, ChestPain.
- ▶ Un árbol podado con validación cruzada da **6 nodos terminales**.



Trees vs. Linear models

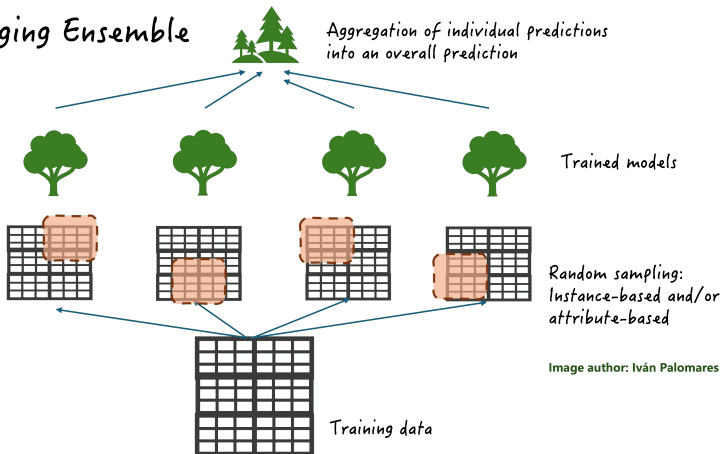


Bagging, Random Forests, and Boosting

Ensemble Methods

- Un **método de ensemble** combina múltiples modelos simples (llamados weak learners) para construir un modelo más robusto y preciso.
- En árboles de decisión, esto permite reducir la **varianza**, un problema común de los árboles individuales.

Bagging Ensemble



- Bagging (**Bootstrap Aggregating**) es una técnica que usa el bootstrap para generar múltiples conjuntos de entrenamiento y luego promedia las predicciones.

Bagging

Promediar reduce la varianza: si $\hat{f}_1(x), \dots, \hat{f}_B(x)$ son predicciones individuales, entonces:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

- ▶ En árboles de regresión, el **bagging** consiste en:
 1. Generar B muestras bootstrap del conjunto de entrenamiento.
 2. Entrenar un árbol de regresión profundo (**sin podar**) para cada muestra.
 3. Promediar las B predicciones individuales.
- ▶ Cada árbol individual tiene **alta varianza** pero **bajo sesgo**.
- ▶ Promediar reduce la varianza, mejorando la precisión global.
- ▶ Se pueden usar **cientos o miles** de árboles sin riesgo de sobreajuste.

- ▶ En problemas de clasificación, la predicción se realiza por **votación mayoritaria**.
- ▶ Para una observación de prueba:
 - Cada uno de los B árboles predice una clase.
 - Se elige la clase más votada como predicción final.
- ▶ La tasa de error de prueba mejora respecto a un solo árbol.
- ▶ El número de árboles B no es sensible: usar un B grande no causa sobreajuste.

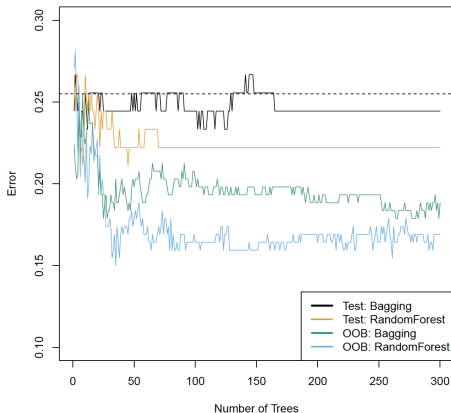
Estimación del error de prueba sin validación cruzada

- ▶ En **bagging**, cada árbol se entrena con una muestra bootstrap ($2/3$ del total).
- ▶ El **$1/3$ restante** de las observaciones se conoce como **Out-of-Bag (OOB)**.
- ▶ Para cada observación i :
 - Se predice su respuesta usando los árboles donde fue OOB.
 - Se obtienen $\sim B/3$ predicciones independientes.
 - Se promedia (regresión) o se vota por mayoría (clasificación).
- ▶ Se calcula el error global comparando las predicciones OOB con los valores reales.
- ▶ El **error OOB** es una estimación válida del error de prueba.

Con B suficientemente grande, el error OOB es casi equivalente al leave-one-out CV.

Results using the dataset Heart

- ▶ Se entrenaron múltiples árboles con distintas muestras bootstrap.
- ▶ El error de prueba disminuye a medida que aumenta el número de árboles B .
- ▶ La curva se estabiliza alrededor de $B = 100$.
- ▶ El bagging logra una **mejor precisión** que un único árbol de decisión.



Error
de prueba vs. número de árboles B usando bagging -
OOB (Out-of-Bag).

El bagging reduce la varianza y mejora el rendimiento predictivo sin riesgo de sobreajuste al aumentar B .

Random Forest

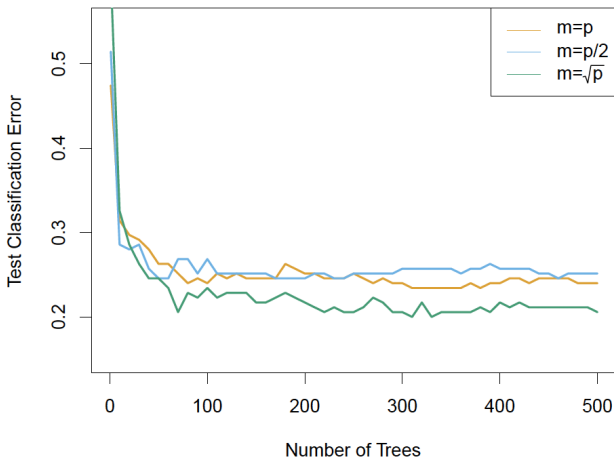
Mejora Bagging con descorrelación de árboles

- ▶ Igual que bagging: entrenamos B árboles sobre subconjuntos bootstrap.
- ▶ Pero: **en cada división de un árbol, se selecciona aleatoriamente un subconjunto de m predictores de los p totales.**
- ▶ El mejor split se elige solo entre estos m predictores.
- ▶ Típicamente, $m \approx \sqrt{p}$ (por ejemplo, $m = 4$ para $p = 13$).
- ▶ Evita que todos los árboles usen el mismo predictor dominante.
- ▶ Los árboles resultantes están menos correlacionados.
- ▶ **Promedia predicciones menos correlacionadas $\rightarrow$ menor varianza $\rightarrow$ mejor generalización.**

Si $m = p$, random forests bagging.

Con datos correlacionados y alto p , usar $m \ll p$ mejora el rendimiento.

Random Forest



- Error de clasificación según número de árboles y valor de m :
 - **Single Tree**: 45.7 % error
 - **Bagging** ($m = p$): mejora respecto a un árbol solo
 - **Random Forest** ($m < p$): ligera mejora sobre bagging
- Valores pequeños de m fomentan diversidad y reducen correlación entre árboles. Esto reduce el error general, mejorando la precisión.

Boosting

Objetivo: Mejorar predicciones combinando múltiples árboles débiles secuencialmente.

► **A diferencia del bagging:**

- Árboles se construyen *uno tras otro*.
- Cada nuevo árbol corrige los **errores del modelo anterior**.

► **No se usa remuestreo bootstrap; se ajusta sobre el mismo conjunto de entrenamiento.**

► Cada árbol se ajusta a los **residuos**:

$$r_i = y_i - \hat{f}(x_i)$$

- Se usan árboles pequeños (*stumps*) para evitar el sobreajuste.
- Se regula el aprendizaje con un **parámetro de contracción** λ (ej., $\lambda = 0,01$ o $0,1$).

Boosting aprende lentamente, pero con mayor precisión...

Algoritmo - Boosting para Árboles de Regresión

Algoritmo (número de árboles B , profundidad d , shrinkage λ):

1. Inicializar predicción: $\hat{f}(x) = 0$
2. Calcular residuos iniciales: $r_i = y_i$
3. Para $b = 1$ hasta B :
 - a) Ajustar árbol $\hat{f}_b$ (con d divisiones) a los residuos r_i
 - b) Actualizar modelo:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}_b(x)$$

- c) Actualizar residuos:

$$r_i \leftarrow r_i - \lambda \hat{f}_b(x_i)$$

4. Resultado final:

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}_b(x)$$

Cada nuevo árbol enfoca el aprendizaje donde el modelo anterior falló.

Boosting tiene tres hiperparámetros principales que deben ajustarse:

1. **Número de árboles B :**

- ▶ Aumentar B mejora el ajuste, pero puede causar *overfitting*.
- ▶ El sobreajuste ocurre lentamente.
- ▶ Se elige B típicamente mediante **validación cruzada**.

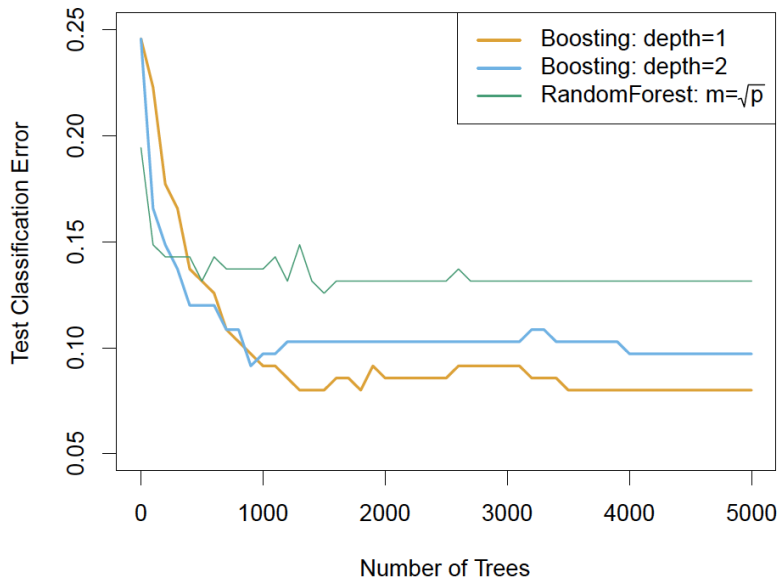
2. **Parámetro de reducción (shrinkage) λ :**

- ▶ Controla la **velocidad de aprendizaje**.
- ▶ Valores típicos: 0,01 o 0,001.
- ▶ Cuanto más pequeño sea λ , más grande debe ser B .

3. **Número de divisiones por árbol d :**

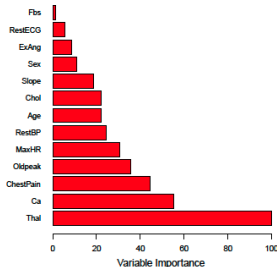
- ▶ Controla la **complejidad** de cada árbol.
- ▶ $d = 1$ produce **stumps** (modelos aditivos).
- ▶ En general, d define la **profundidad de interacción**.

Boosting



Variable Importance in Bagging

- ▶ Bagging mejora la **precisión predictiva** respecto a un único árbol, pero sacrifica la interpretabilidad².
- ▶ Para evaluar la **importancia de variables**:
 - **Regresión**: se mide la reducción total del RSS por variable, promediada en todos los B árboles.
 - **Clasificación**: se mide la reducción total del índice de Gini por variable, promediada en todos los B árboles.
- ▶ Una **gran disminución** en RSS o Gini indica un predictor importante.
- ▶ Ejemplo en *Heart Data*: las variables más relevantes son **Thal**, **Ca** y **Chest-Pain**.



²Un árbol individual puede representarse fácilmente con un diagrama, pero una colección de árboles bagged no.

Variable Importance in Boosting

- ▶ Boosting construye árboles secuenciales, corrigiendo errores de los anteriores.
- ▶ La importancia de cada predictor se mide por la **reducción total** en la función de pérdida (RSS en regresión, Gini o devianza en clasificación).
- ▶ Se acumula la contribución de cada split en todos los árboles del ensamble.
- ▶ Una variable es **más importante** si produce mayor reducción acumulada en la pérdida.
- ▶ En boosting, los árboles suelen ser poco profundos, por lo que destacan variables que realmente ayudan a mejorar la predicción.

Bayesian Additive Regression Trees (BART)

- ▶ Ensemble bayesiano basado en árboles de regresión como bloques.
- ▶ Relación con otros métodos:
 - **Bagging/Random Forests:** árboles independientes, promedio de predicciones.
 - **Boosting:** emplea una suma ponderada de árboles, donde cada árbol se ajusta a los residuos del modelo actual. Así, cada nuevo árbol intenta capturar la señal que aún no ha sido explicada por el conjunto de árboles existente.
- ▶ BART está relacionado con ambos enfoques. **La novedad de BART radica en la forma en que se generan los nuevos árboles.**
- ▶ Generar árboles de manera bayesiana, con perturbaciones aleatorias para evitar sobreajuste.

La notación $\hat{f}_k^b(x)$ representa la predicción en x del k -ésimo árbol de regresión en la b -ésima iteración. Al final de cada iteración, los K árboles de esa iteración se suman:

$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x), \quad b = 1, \dots, B. \quad (1)$$

En la primera iteración del algoritmo BART, todos los árboles se inicializan con un único nodo raíz, con

$$\hat{f}_k^1(x) = \frac{1}{nK} \sum_{i=1}^n y_i,$$

Así,

$$\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = \frac{1}{n} \sum_{i=1}^n y_i.$$

En las iteraciones posteriores, BART actualiza cada uno de los K árboles de forma secuencial. En la b -ésima iteración, el k -ésimo árbol se ajusta a partir del residuo parcial, definido como³:

$$r_i = y_i - \sum_{k' < k} \hat{f}_{k'}^b(x_i) - \sum_{k' > k} \hat{f}_{k'}^{b-1}(x_i), \quad i = 1, \dots, n.$$

³Los símbolos $<$ y $>$ en la fórmula indican qué árboles ya han sido actualizados en esta iteración y cuáles no.

BART selecciona **aleatoriamente** una perturbación del árbol de la iteración anterior $\hat{f}_k^{b-1}$ a partir de un conjunto de posibles perturbaciones, favoreciendo aquellas que mejoran el ajuste al residuo parcial.

Existen dos componentes principales en esta perturbación:

1. Se puede modificar la estructura del árbol añadiendo o podando ramas.
2. Se pueden cambiar las predicciones en cada nodo terminal del árbol.

La salida de BART es un conjunto de modelos de predicción, dado por

$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x), \quad b = 1, 2, \dots, B.$$

Algoritmo BART

1. Inicialización: $\hat{f}_1^1(x) = \hat{f}_2^1(x) = \dots = \hat{f}_K^1(x) = \frac{1}{nK} \sum_{i=1}^n y_i$

$$\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = \frac{1}{n} \sum_{i=1}^n y_i$$

2. Para $b = 2, \dots, B$:

1) Para $k = 1, \dots, K$: **Recorrer los árboles uno a uno.**

1) Calcular el residuo parcial:

$$r_i = y_i - \sum_{k' < k} \hat{f}_{k'}^b(x_i) - \sum_{k' > k} \hat{f}_{k'}^{b-1}(x_i), \quad i = 1, \dots, n$$

2) Ajustar un nuevo árbol $\hat{f}_k^b(x)$ perturbando el árbol previo $\hat{f}_k^{b-1}(x)$ **(no se rehace de cero)⁴**.

2) Actualizar el ensamble:

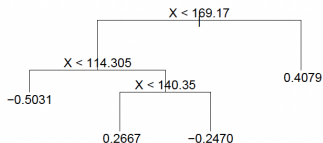
$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x)$$

3. Estimación final (tras L iteraciones de *burn-in*):

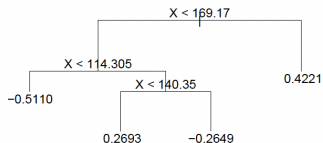
$$\hat{f}(x) = \frac{1}{B-L} \sum_{b=L+1}^B \hat{f}^b(x)$$

⁴añadir una división, podar, cambiar un umbral de división, intercambiar divisiones, y/o ajustar los valores en las hojas.)

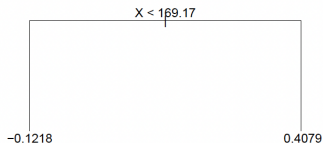
(a): $\hat{f}_k^{b-1}(X)$



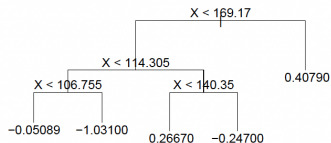
(b): Possibility #1 for $\hat{f}_k^b(X)$



(c): Possibility #2 for $\hat{f}_k^b(X)$



(d): Possibility #3 for $\hat{f}_k^b(X)$



Esquema de posibles perturbaciones en los árboles del algoritmo BART: (b) cambio en los nodos terminales, (c) poda, (d) crecimiento.

Componentes clave de BART

- ▶ **Partial residual**, cada árbol se ajusta solo al error que no explican los demás. Los priors favorecen árboles poco profundos y valores pequeños en las hojas (shrinkage).
- ▶ **Burn-in period**: primeras iteraciones no se usan por baja estabilidad.
- ▶ **Salida**: conjunto de modelos $\hat{f}^{(b)}(x)$ que permite cuantificar incertidumbre.
- ▶ BART asigna distribuciones a priori a la estructura de los árboles y a los valores en los nodos terminales, lo que induce regularización y evita sobreajuste.
- ▶ La predicción se construye como la suma de muchos árboles pequeños y débiles, cada uno aportando una parte de la señal.
- ▶ **Aplicaciones**: Especialmente útil en regresión no lineal, problemas con interacciones complejas y cuando se requieren medidas confiables de incertidumbre.

Métodos que combinan modelos débiles para formar un modelo fuerte.

- ▶ AdaBoost
- ▶ Gradient Boosting
- ▶ LightGBM CatBoost
- ▶ XGBoost



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Aprendizaje automático - Sesión 6

Bagging and Boosting

Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.
`jorge.portilla@usm.cl`

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile